Carátula para entrega de prácticas

Laboratorio de computación salas A y B

Profesor:	M.C Rene Adrian Davila Perez
Asignatura:	Programación orientada a objetos
Grupo:	7
No de Práctica(s):	10
Integrante(s):	323036285 323279866 323209432
	323161891 323063609
No. De Lista o brigada	3
Semestre:	2027-1
Fecha de entrega:	6/11/2026
Obervaciones:	

CALIFICACIÓN: _____

Índice

1. Introducción **2**

2. Marco Teórico **3**
2.1. Concepto de Excepción y Bloques de Control 3
2.2. Jerarquía de la Clase `java.lang.Throwable` 3
2.3. Excepciones Comprobadas (Checked) y No Comprobadas (Unchecked) 4
2.4. Captura Polimórfica mediante la Superclase `Exception` 4

3. Desarrollo **5**
3.1. Arquitectura y Funcionamiento de la Clase `Division` 5
3.2. Flujo de Control y Captura Polimórfica en la Clase `Main` . . . 6

4. Resultados **8**
4.1. Ejercicio de IA 8
4.1.1. Jerarquía de `Throwable` en Java: `Error` y `Exception` . 8
4.2. Pruebas de validación 11
4.2.1. Prueba 1: Ejecución correcta 11
4.2.2. Prueba 2: Número negativo 12
4.2.3. Prueba 3: División entre cero 12
4.2.4. Prueba 4: Entrada no numérica 13

5. Conclusión **14**

1. Introducción

Al iniciar en el lenguaje de programación Java, al estudiantado se le presentan dificultades al enfrentarse a fallas inesperadas y datos inválidos durante la ejecución de los programas, lo que puede corromper el estado de los objetos y frustrar el manejo de errores ante la falta de mecanismos adecuados de control.

Resolver este problema nos ayudará a comprender cómo capturar distintos errores simultaneamente usando un solo bloque catch, a validar los datos desde el constructor para evitar que diferentes situaciones dañen el programa, y a asegurar el cierre correcto de los recursos de lectura en consola mediante finally.

Objetivos

- Realizar la captura de múltiples excepciones al ejecutar un programa, mediante la instrucción catch, para comprender el funcionamiento de la sustitución polimórfica en el control de fallas.
- Aplicar un buen encapsulamiento y la validación de argumentos en el constructor de una clase, para evitar datos que corrompan al sistema posteriormente.
- Garantizar la liberación incondicional de los recursos del flujo de lectura de la consola mediante la cláusula finally, para asegurar que los componentes del sistema se cierren de forma ordenada tras la ejecución del programa.

2. Marco Teórico

2.1. Concepto de Excepción y Bloques de Control

Una excepción en Java es un objeto que representa una condición anómala suscitada durante la ejecución de un programa, la cual interrumpe el curso ordinario de las instrucciones para transferir el control a un bloque especializado de atención [1]. El tratamiento de estos eventos se estructura a través de tres componentes principales [4]:

- **Bloque `try`:** Delimita el conjunto de instrucciones vigiladas que son susceptibles de presentar fallas durante su procesamiento.
- **Bloque `catch`:** Declara el tipo de excepción a interceptar y contiene las acciones necesarias para atender el problema y evitar la terminación abrupta del programa.
- **Bloque `finally`:** Se ejecuta de manera incondicional tanto si el bloque protegido concluyó satisfactoriamente como si se produjo una excepción, constituyendo el espacio designado para el cierre de flujos y la liberación de recursos [4].

2.2. Jerarquía de la Clase `java.lang.Throwable`

En Java, la base de la jerarquía de errores y excepciones reside en la clase `Throwable`, de la cual se desprenden dos ramas fundamentales e independientes [3]:

- **Rama `Error`:** Agrupa condiciones severas de fallo originadas en el entorno del sistema operativo o en la propia Máquina Virtual de Java, tales como el agotamiento de memoria física (`OutOfMemoryError`) o el desbordamiento de la pila de llamadas (`StackOverflowError`). Por diseño, los programas convencionales no deben intentar capturar ni recuperarse de estas fallas críticas [3].
- **Rama `Exception`:** Modela condiciones de error razonables de las cuales una aplicación sí puede y debe recuperarse para mantener su ejecución continua [1].

2.3. Excepciones Comprobadas (Checked) y No Comprobadas (Unchecked)

Las excepciones derivadas de `Exception` se dividen en dos categorías según la forma en que el compilador exige su tratamiento [1, 2]:

- **No comprobadas (Unchecked / `RuntimeException`):** Subclases directas o indirectas de `java.lang.RuntimeException`, como `ArithmeticException`, `IllegalArgumentException` o `NumberFormatException`. Representan descuidos lógicos o datos incorrectos que pueden prevenirse mediante programación defensiva; el compilador no exige confinarlas en un bloque `try-catch` ni declararlas forzosamente [2].
- **Comprobadas (Checked):** Todas aquellas subclases de `Exception` que no descienden de `RuntimeException`, como `IOException`. El compilador verifica rigurosamente su presencia, demandando su captura forzosa mediante `try-catch` o su delegación en la firma del método con la palabra reservada `throws` [1].

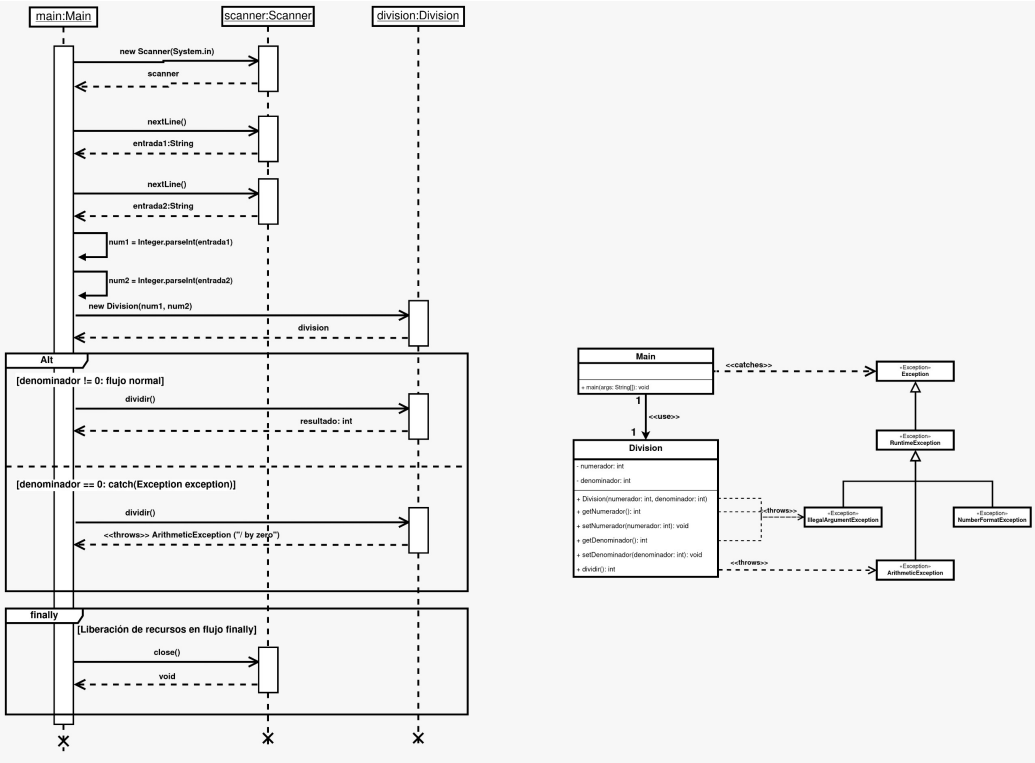
2.4. Captura Polimórfica mediante la Superclase `Exception`

Debido a la jerarquía de herencia y al polimorfismo, una cláusula `catch` que declara un parámetro de tipo `Exception` posee la capacidad de atrapar cualquier objeto perteneciente a sus ramas derivadas [4, 3]. Esto abarca tanto excepciones comprobadas como no comprobadas. No obstante, este bloque no puede capturar instancias de la clase `Error`, ya que esta última forma una rama independiente dentro de `Throwable` [3].

3. Desarrollo

El programa se estructuró de forma modular mediante dos clases independientes: la clase `Division`, encargada de modelar la operación matemática y sus restricciones, y la clase `Main`, responsable de interactuar con el usuario a través de la terminal y de gestionar la captura polimórfica de excepciones.

Para realizar el programa se siguió el siguiente UML.



3.1. Arquitectura y Funcionamiento de la Clase `Division`

La clase `Division` se implementó aplicando encapsulamiento estricto. Se declararon dos atributos enteros privados denominados `numerador` y `denominador`, impidiendo su acceso directo desde otras clases y asegurando que su consulta y actualización se realizaran por medio de métodos específicos.

Para resguardar el estado de los objetos, el constructor de la clase se definió recibiendo los dos valores enteros y declarando explícitamente en su firma la cláusula `throws IllegalArgumentException`. Dentro de este cons-

tructor se implementó una estructura condicional que evaluó si el numerador o el denominador eran menores a cero. Al presentarse una cantidad negativa, la instanciación se interrumpió de inmediato y se lanzó una excepción `IllegalArgumentException` con el mensaje indicando que no se permiten números negativos. Si ambos datos cumplieron con ser no negativos, se asignaron a los atributos correspondientes mediante la referencia `this`. Es importante señalar que el constructor admitió el valor de cero en el denominador, dejando la evaluación de la división indeterminada para el método de cálculo.

Asimismo, se codificaron métodos de lectura para obtener el valor de cada atributo y métodos de modificación que incorporaron la misma validación defensiva del constructor, lanzando `IllegalArgumentException` si se intentaba asignar un número negativo. Por último, se desarrolló el método `dividir`, el cual declaró en su encabezado la cláusula `throws ArithmeticException` y calculó el cociente entero de la operación. En este método se omitió a propósito una estructura condicional que verificara si el divisor era cero, permitiendo que la Máquina Virtual de Java detectara la división entera entre cero en tiempo de ejecución y produjera de forma natural la excepción `ArithmeticException`.

3.2. Flujo de Control y Captura Polimórfica en la Clase `Main`

La clase `Main` albergó el punto de inicio de la aplicación y gestionó la interacción con la consola a través de un objeto de la clase `Scanner`. Toda la lógica de lectura de datos, conversión numérica, creación del objeto e invocación del cálculo se agrupó dentro de un bloque `try` para proteger el flujo del programa.

La ejecución comenzó solicitando al usuario el numerador en la consola, leyendo la entrada como una cadena de texto para procesarla con el método `Integer.parseInt`. Si el usuario ingresó caracteres alfanuméricos o texto inválido, este método de conversión falló de inmediato y produjo una excepción de tipo `NumberFormatException`, suspendiendo la ejecución del bloque protegido en ese instante sin llegar a solicitar el denominador. Si la conversión del numerador fue exitosa, se procedió a solicitar y convertir el denominador bajo el mismo procedimiento.

Una vez obtenidos ambos valores enteros, se instanció la clase `Division`. Si se proporcionó un número negativo, el constructor lanzó la excepción

`IllegalArgumentException`. Al crearse el objeto con datos válidos, se invocó el método `dividir`, el cual produjo la excepción `ArithmeticException` si el denominador ingresado fue cero. En caso de no presentarse ninguna anomalía, se imprimió en consola el resultado del cociente obtenido.

Para atender cualquier fallo en la ejecución, se implementó una única cláusula `catch` declarando un parámetro de tipo `Exception`. Al suscitarse cualquiera de las tres posibles excepciones, el flujo de ejecución se transfirió a este manejador polimórfico, donde se invocó el método `getClass().getSimpleName()` para obtener el nombre exacto de la clase de error ocurrida y se extrajo el mensaje descriptivo de la falla mediante `getMessage()`. Finalmente, se ejecutó de forma obligatoria la cláusula `finally`, en la cual se mostró en pantalla un mensaje indicando que el código había concluido y se invocó el método `close()` del objeto `Scanner` para cerrar adecuadamente el flujo de lectura de la consola.

4. Resultados

4.1. Ejercicio de IA

Con el siguiente *prompt* le pedimos a la IA las clases de la jerarquía `Error` y las excepciones más comunes en una GUI.

> Estamos en programación orientada a objetos, soy estudiante de ingeniería de tercer semestre de la carrera de ingeniería en telecomunicaciones dime las clases de la jerarquía `Error` en Java y las excepciones más comunes en una GUI.

4.1.1. Jerarquía de `Throwable` en Java: `Error` y `Exception`

Estructura general Todo lo que se puede "lanzar" (`throw`) en Java desciende de `java.lang.Throwable`:

```
Object
  Throwable
       Error
           VirtualMachineError
               OutOfMemoryError
               StackOverflowError
               InternalError
           LinkageError
               NoClassDefFoundError
               ExceptionInInitializerError
               UnsatisfiedLinkError
           AssertionError
           ThreadDeath

       Exception
            IOException
                FileNotFoundException
                EOFException
            SQLException
            ClassNotFoundException
            InterruptedException
            RuntimeException
```

```
NullPointerException
ArithmeticException
ArrayIndexOutOfBoundsException
ClassCastException
IllegalArgumentException
    NumberFormatException
IllegalStateException
```

La rama `Error` Los `Error` representan fallas graves de la JVM o del entorno de ejecución. No se espera que el programa los atrape ni se recupere de ellos; normalmente indican que algo anda muy mal.

Ejemplo clásico de `StackOverflowError`:

```
public static int infinito(int n) {
    return infinito(n + 1);
}
```

Checked vs. Unchecked

Excepciones más comunes en una GUI (Swing / JavaFX) `NumberFormatException`: el usuario escribe texto donde se espera un número en un `JTextField`.

```
try {
    int edad = Integer.parseInt(txtEdad.getText());
} catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(this,
        "Ingresa un número válido");
}
```

- `NullPointerException`: usar un componente no inicializado o un valor nulo.
- `ArithmeticException`: división entre cero cuando los datos vienen de campos de texto.
- `IllegalArgumentException`: valores inválidos para un método.
- `ArrayIndexOutOfBoundsException` e `IndexOutOfBoundsException`: acceder a una fila inexistente de una `JTable` o a un índice inválido en un `JList` o `ArrayList`.

Error	**Cuándo ocurre**
`OutOfMemoryError`	La JVM se quedó sin memoria heap (por ejemplo, crear arreglos gigantes o fugas de memoria).
`StackOverflowError`	Recursión infinita o demasiado profunda.
`NoClassDefFoundError`	La clase existía al compilar pero no se encuentra al ejecutar.
`ExceptionInInitializerError`	Falla dentro de un bloque `static` o inicializador estático.
`UnsatisfiedLinkError`	No se encuentra una librería nativa (métodos `native`).
`AssertionError`	Falla una instrucción `assert`.
`InternalError`	Error inesperado interno de la JVM.

Cuadro 1: Errores representativos de la jerarquía `Error`.

Tipo	**Clases**	**¿Obliga el compilador**
Checked	`Exception` y sus subclases, excepto `RuntimeException`	Sí: `try-catch` o `throws`
Unchecked	`RuntimeException` y subclases, y todos los `Error`	No

Cuadro 2: Comparación entre excepciones checked y unchecked.

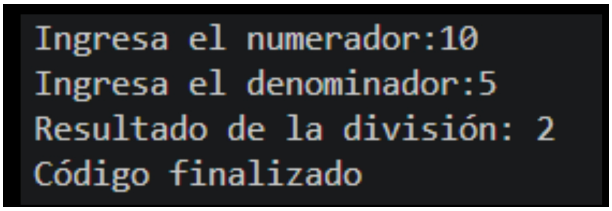
- `ClassCastException`: convertir incorrectamente un objeto al recuperar `e.getSource()` en un evento.
- `IOException` y `FileNotFoundException`: al abrir o guardar archivos con `JFileChooser`.
- `SQLException`: al conectar la GUI con una base de datos.
- `ClassNotFoundException`: al cargar un driver mediante `Class.forName(...)`.
- `IllegalStateException`: usar un componente en un estado inválido.
- `HeadlessException`: intentar crear una ventana en un entorno sin pantalla.
- `InterruptedException`: al utilizar `Thread.sleep()` o hilos dentro de la GUI.
- `IllegalComponentStateException`: realizar operaciones sobre componentes que aún no se muestran correctamente.
- En JavaFX es común el error `IllegalStateException: Not on FX application thread`, producido al modificar la interfaz desde un hilo distinto al hilo principal de JavaFX.

4.2. Pruebas de validación

Tras concluir el desarrollo del código, se realizaron las pruebas de validación necesarias para asegurar los resultados y verificar el comportamiento de la captura polimórfica. Se ejecutó el archivo independiente de la práctica evaluando cuatro escenarios distintos desde la terminal de comandos.

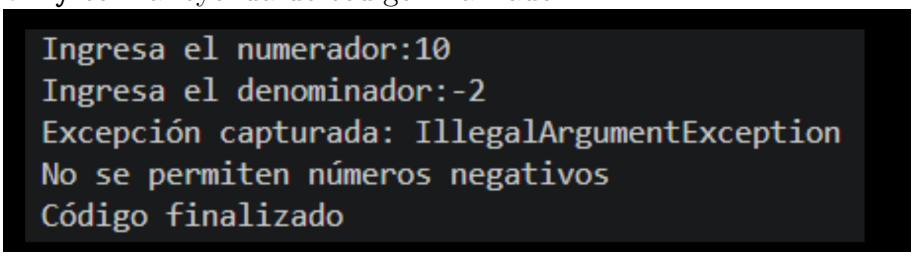
4.2.1. Prueba 1: Ejecución correcta

En la primera prueba se ingresaron valores numéricos enteros positivos válidos, asignando un valor de 10 al numerador y un valor de 2 al denominador. Las conversiones y la creación de la clase `Division` se procesaron de forma regular, desplegando en la consola el cociente obtenido con un valor de 5. Posteriormente, la ejecución pasó de manera directa al bloque `finally`, donde se mostró la leyenda de finalización y se cerró el flujo del lector de la terminal.



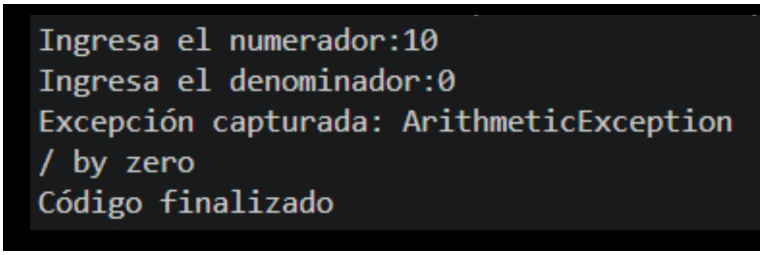
4.2.2. Prueba 2: Número negativo

En la segunda prueba se evaluó el manejo de valores numéricos negativos, capturando un valor de 10 en el numerador y un valor de -2 en el denominador. Las cadenas se convirtieron a enteros sin problema, pero al inicializar el objeto, el constructor de la clase `Division` detectó el argumento negativo y lanzó la excepción `IllegalArgumentException`. El bloque `catch` interceptó el evento e imprimió en consola la etiqueta "Excepción capturada: IllegalArgumentException" seguida en la siguiente línea del mensaje "No se permiten números negativos", concluyendo la ejecución de forma limpia en la cláusula `finally` con la leyenda de código finalizado.



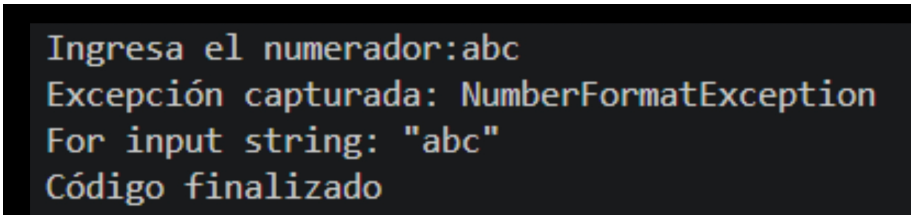
4.2.3. Prueba 3: División entre cero

En la tercera prueba se suministró un valor de 10 en el numerador y un valor de 0 en el denominador. El constructor creó el objeto satisfactoriamente al ser ambos números no negativos, pero al invocar el método `dividir`, la Máquina Virtual de Java detectó la división entre cero y produjo la excepción `ArithmeticException`. La cláusula `catch` capturó oportunamente el problema y desplegó en la consola la etiqueta "Excepción capturada: ArithmeticException" junto con el mensaje "/ by zero", ejecutando enseguida el bloque `finally` sin que el programa se detuviera de manera imprevista.



4.2.4. Prueba 4: Entrada no numérica

En la cuarta prueba se introdujo una cadena en el numerador ingresando el texto "abc". Al ejecutarse el método `Integer.parseInt`, la conversión numérica falló instantáneamente y produjo la excepción `NumberFormatException`. Debido a esto, la ejecución dentro del bloque protegido se suspendió de inmediato, transfiriendo el control al bloque `catch` sin solicitar ni leer el valor del denominador. En la terminal se mostró la salida "Excepción capturada: NumberFormatException" acompañada del mensaje nativo "For input string: .ªbc', transitando posteriormente a la cláusula `finally` para cerrar el flujo de entrada.



5. Conclusión

Tras realizar la práctica se concluyó que el polimorfismo facilita el control de fallas al capturar distintas excepciones con un solo bloque catch, mientras que la validación de argumentos en el constructor protege la integridad de los objetos al impedir que datos no válidos corrompan el sistema. Asimismo, se analizó que la cláusula finally resulta imprescindible para la arquitectura del programa, ya que garantiza la liberación incondicional y ordenada de recursos como el flujo de lectura de la consola, asegurando un diseño de software robusto, preventivo y eficiente.

Referencias

[1] Paul J. Deitel y Harvey M. Deitel. *Java Cómo Programar*. pp. 441–470. México, 2016.

[2] GeeksforGeeks. *Exceptions in Java*. `https://www.geeksforgeeks.org/exceptions-in-java/`. Accedido: 04-nov-2026.

[3] Oracle Corporation. *Class Exception*. `https://docs.oracle.com/javase/8/docs/api/java/lang/Exception.html`. Java Platform, Standard Edition 8 API Specification. Accedido: 04-nov-2026. 2014.

[4] Oracle Corporation. *Lesson: Exceptions*. `https://docs.oracle.com/javase/tutorial/essential/exceptions/`. The Java Tutorials. Accedido: 04-nov-2026.